Artificial Intelligence
Introduction and examples

Stefaan Haspeslagh
Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

Artificial Intelligence: introduction and examples

OPTIMAL SEARCH

Optimal search

- When cost of TRAVERSING the path should be minimized (even at expense of more complicated SEARCHING)

- Some algorithms:
  - Uniform cost
  - Branch and bound extention
  - Heuristic estimate extension
  - A*

In de vorige topics zochten we ofwel ‘een pad’ naar het doel, ofwel het ‘kortste pad’ naar het doel.

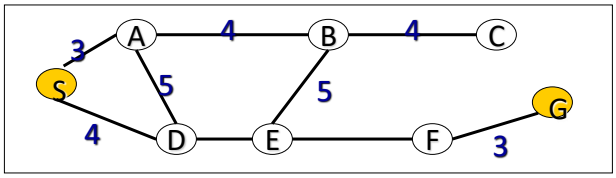
Met kortste pad bedoelen we het pad langs het minste aantal knopen.

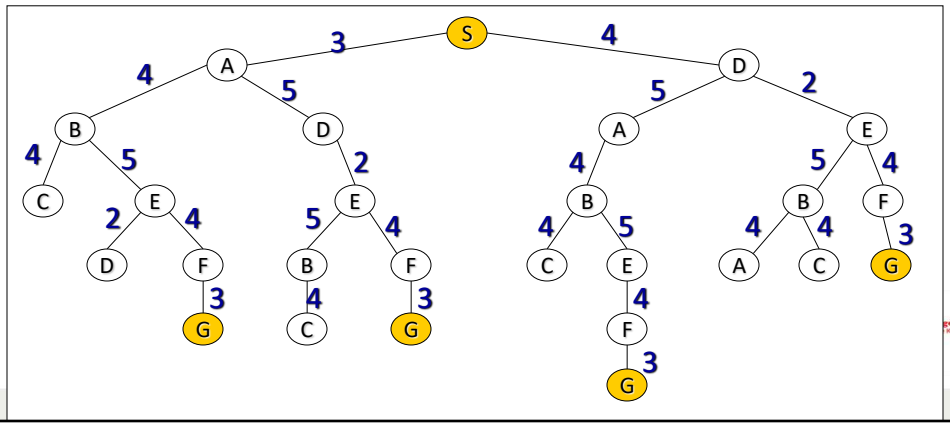
Vanaf nu willen we het meest optimale pad vinden, dit is het pad met de minste kost.

We bestuderen eerst het uniform cost algoritme en verschillende uitbreidingen ervan. Zo komen we uiteindelijk tot het A*-algoritme waarvan we de werking ook zullen uitleggen.

Uniform cost algorithm

- From now on, we use the costs of arcs in a NET or search tree





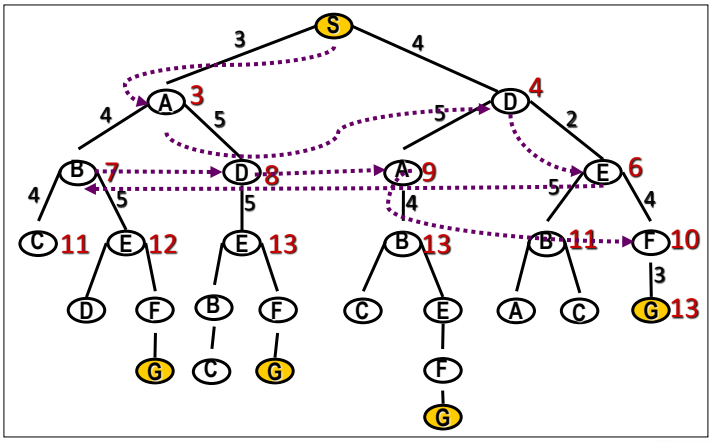
Zoals in de vorige slide aangegeven, gaan we vanaf nu rekening houden met de kosten van een boog in een netwerk of van een tak in een boom.

Kosten kunnen bijvoorbeeld aanzien worden als afstanden. De afstand tussen knoop S en knoop A is bijvoorbeeld 3 km, de afstand tussen knoop S en knoop D is 4 km. Het zoeken van het optimale pad van S naar G (het doel) komt dan neer op het zoeken van de ‘weg’ van S naar G met de kortste afstand.

De algoritmen die we in dit hoofdstuk zien, en dan vooral het A*-algoritme, kunnen effectief in de praktijk gebruikt worden voor een eenvoudige routeplanner.

Uniform cost algorithm

- At each step, select the node with the lowest accumulated cost.



Het uniform cost algoritme is gebaseerd op het volgende idee en principe.

Om het optimale pad te vinden zou je kunnen verwachten dat je altijd best het pad met de laagste kost in de QUEUE moet selecteren. Naïef gezien zou dit er inderdaad misschien voor kunnen zorgen dat het optimale pad gevonden wordt.

Op de figuur is een voorbeeld van dit algoritme uitgewerkt.

Eerst zit enkel S in de QUEUE.
Dit is dus een pad met kost 0. Dit pad wordt geselecteerd.

De kinderen van S worden gegenereerd, dit zijn dus SA (3) en SD (4), en toegevoegd aan de QUEUE. Tussen haakjes staan de kosten van de paden.

SA is het pad met de laagste kost, dus wordt dit geselecteerd en worden de kinderen ervan gegenereerd, SAB (7) en SAD (8), en toegevoegd aan de QUEUE.
(De kost van een pad tot aan een knoop wordt in de figuur in rood bij de knoop geschreven.)

Er zitten op dit moment drie paden in de QUEUE: SAB (7), SAD (8) en SD (4).
SD is het pad in de QUEUE met de laagste kost, dus dit pad wordt geselecteerd en de kinderen worden gegenereerd en toegevoegd aan de QUEUE: SDA (9) en SDE (6).

Het algoritme gaat zo verder tot een pad dat naar doel G leidt gevonden wordt.

Probeer zelf de volgende stappen uit te schrijven en het pad tot aan het doel te vinden.

Ter verduidelijking: met accumulated kost van een pad wordt de totale kost van het pad bedoeld. Soms wordt dit verward met de kost van de laatste ‘link’ op het pad. Voorbeeld:

- De geaccumuleerde kost van pad SAD is 8
- De kost van de laatste link op het pad SAD is 5

Uniform cost algorithm

1. QUEUE ←path only containing the root;

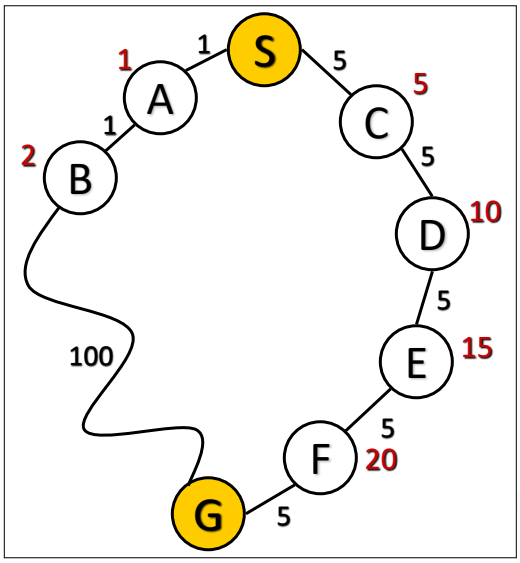
2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the QUEUE and sort the entire QUEUE (by accumulated cost)

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

Het algoritme lijkt sterk op greedy search. Maar in plaats van een sortering op heurstieke waarde f (zie topic 6), wordt er gesorteerd op geaccumuleerde kost van het pad.

Uniform cost algorithm

- Uniform cost returns the path with cost 102, while there is a path with cost 25.



Ter herinnering: uniform cost neemt telkens het pad uit de QUEUE met de laagste kost, genereert daarvan de kinderen, voegt ze toe aan de QUEUE, en sorteert de volledige QUEUE op kost.
Dit gaat zo verder tot alle paden bekeken zijn of tot een pad naar het doel gegenereerd werd.

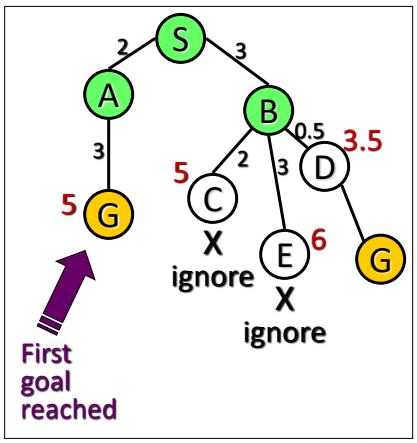
Als we het algoritme op de zoekstructuur op deze slide toepassen, dan bekomen we de volgende toestanden van de QUEUE.
1) S (0)
2) SA (1), SC (5)
3) SAB (2), SC (5)
4) SC (5), SABG (102)

Vanaf nu zit er een pad naar het doel in de QUEUE en is dus niet meer voldaan aan de voorwaarde van de while-loop.
Het algoritme stopt en heeft dus het pad met lengte 102 gevonden.

We zien dus onmiddellijk al het probleem met het uniform cost algoritme opduiken: uniform cost vindt niet noodzakelijk het optimale pad. Ons naïef criterium om telkens het pad met de laagste kost te selecteren blijkt toch niet zo uitmuntend te zijn...

The branch-and-bound principle

- Use any (complete) search method to find a path.
- Remove all partial paths that have an accumulated cost larger or equal than the found path.
- Continue search for the next path.
- Iterate.



Met branch-and-bound probeert men het probleem met het vorige algoritme aan te pakken.

We kunnen stellen dat het vorige algoritme te vroeg stopt met zoeken. Hadden we een beetje verder gezocht, dan hadden we het pad naar het doel van kost 25 gevonden.

Gevaar met verder zoeken is natuurlijk dat de rekentijd zal toenemen. We proberen hier een evenwicht in te vinden door bepaalde takken van de boom te snoeien.

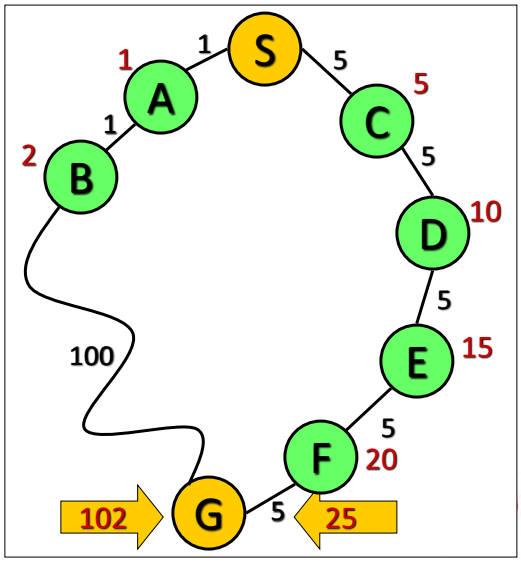
Het idee is als volgt: van zodra een pad naar het doel gevonden is (dit is net zoals bij uniform cost niet noodzakelijk het optimale pad!) kan men beginnen snoeien.
De kost van het tot nu toe gevonden pad is een bovengrens (Engels: upper bound!) voor het optimale pad dat kan gevonden worden. We noemen die kost *kleinst_gevonden_kost*
Dus paden die we tegenkomen tijdens de analyse van de boom, die langer worden dan het getal *kleinst_gevonden_kost* moeten niet meer onderzocht worden. Want die paden zullen toch nooit een kleinere kost hebben dan *kleinst_gevonden_kost.*

Een voorbeeld: stel dat in de figuur op de slide, het pad SAG reeds gevonden werd. Dan is het getal *kleinst_gevonden_kost* = 5.
Op een bepaald moment worden de kinderen van B gegenereerd en toegevoegd aan

de QUEUE. Dan worden de paden SBC (5), SBE (6) en SBD (3,5) gegenereerd. Op dat moment kan je eigenlijk beslissen dat SBC en SBE nooit kunnen evolueren tot een pad met lagere kost dan *kleinst_gevonden_kost*, wat tot nu toe 5 is.
Zo zie je eigenlijk dat je de branching factor van knoop B, die 3 is door het branch-and-bound principe, gereduceerd is tot 1. Dit kan dus op gebied van rekentijd een groot verschil maken!

Optimal uniform cost

- A weak integration of branch and bound in the uniform cost algorithm
- Change the termination condition:
  - only terminate when a path to a goal node HAS BECOME THE BEST PATH.



Het optimal uniform cost algoritme is het gewone uniform cost algoritme uitgebreid met een deel van het branch-and-bound principe:

- Net als bij branch-and-bound blijven we verder zoeken, niet oneindig, maar tot het pad naar het doel eerst komt te staan in de QUEUE.
- Het weggooien van paden gebeurt echter NIET in optimal uniform cost.

Optimal uniform cost algorithm

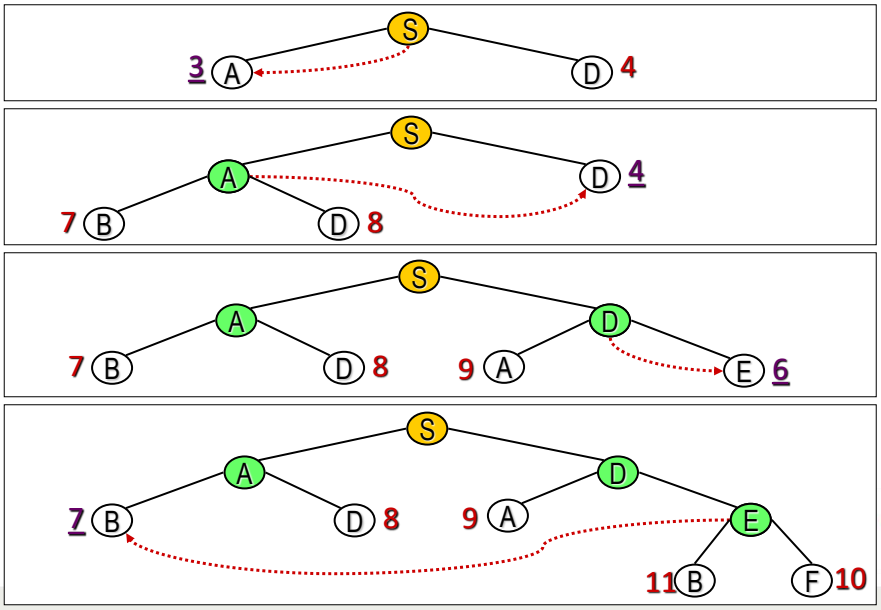
1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND first path does not reach goal)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the QUEUE and sort the entire QUEUE (by accumulated cost)

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

In geel zien we mooi dat het stopcriterium “goal is not reached” wordt vervangen door “first path does not reach goal”. Dat zal er voor zorgen dat we blijven verder zoeken.

Optimal uniform cost algorithm: example

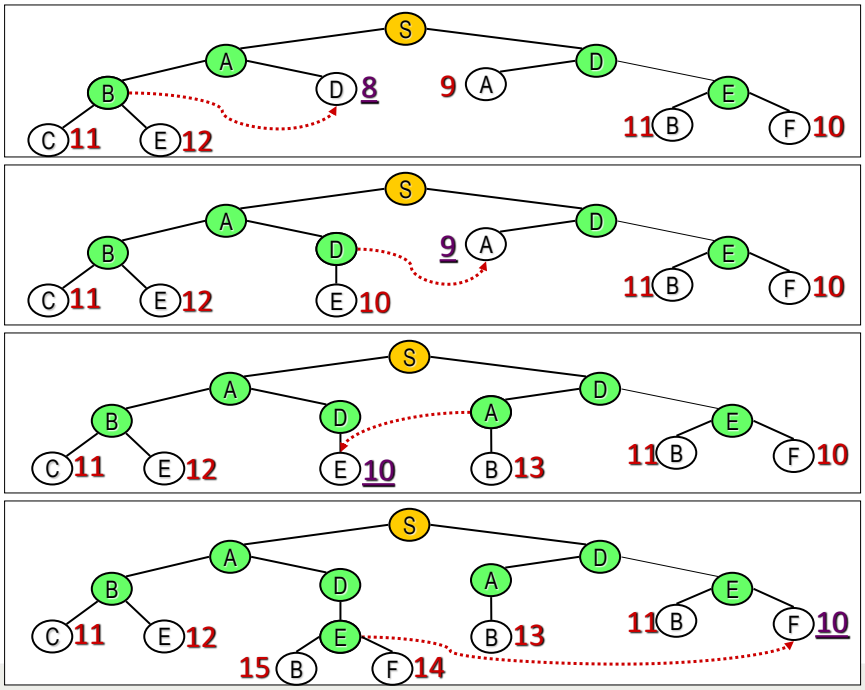


Op deze en de volgende twee slides wordt een voorbeeld uitvoering van het optimal uniform cost algoritme.

Opeenvolgende toestanden van de QUEUE, telkens na sortering:

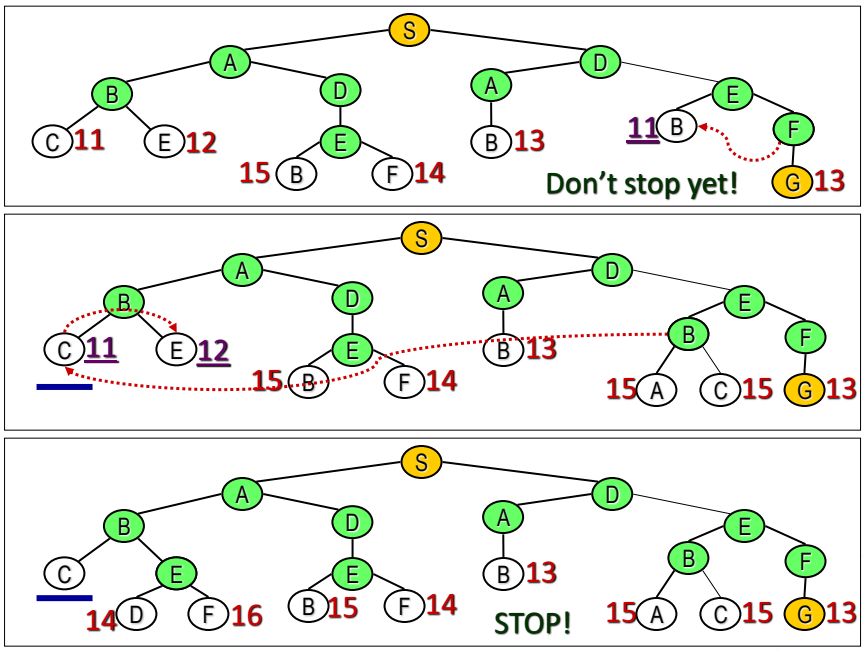
- SA (3), SD (4)
- SD (4), SAB (7), SAD (8)
- SDE(6), SAB (7), SAD (8), SDA (9)
- SAB (7), SAD (8), SDA (9), SDEF (10), SDEB (11)

Optimal uniform cost algorithm example



- SAD (8), SDA (9) , SDEF (10), SABC (11), SDEB (11), SABE (12)
- SDA (9) , SADE (10), SDEF (10), SABC (11), SDEB (11), SABE (12)
- SADE (10), SDEF (10), SABC (11), SDEB (11), SABE (12), SDAB (13)
- SDEF (10), SABC (11), SDEB (11), SABE (12), SDAB (13), SADEF (14), SADEB (15)

Optimal uniform cost algorithm example



- SABC (11), SDEB (11), SABE (12), SDAB (13), SDEFG (13), SADEF (14), SADEB (15)
- Let op, vanaf de vorige stap werd een pad naar het doel gevonden, maar dat pad is niet het eerste in de QUEUE, dus we stoppen niet!
- SDEB (11), SABE (12), SDAB (13), SDEFG (13), SADEF (14), SADEB (15)
- Vanuit het pad SABC heeft geen verdere mogelijkheden zonder een loop te genereren, dus er wordt niets aan de QUEUE toegevoegd
- SABE (12), SDAB (13), SDEFG (13), SADEF (14), SADEB (15), SDEBA (15), SDEBC (15)
- SDAB (13), SDEFG (13), SABED (14), SADEF (14), SADEB (15), SDEBA (15), SDEBC (15), SABEF (16)

Het pad SDEFG zit nu eerst in de QUEUE (dit is het pad naar de goal met de laagste geaccumuleerde kost) dus we stoppen!

Optimal uniform cost properties

- Complete
  - Yes, mostly
  - Can find the optimal path if one exists
- Memory and speed
  - In the worst case, at least as bad as for breadth-first
    - Needs additional sorting step after each path-expansion
- How to improve?

Voor dit algoritme is het voldoende dat je weet dat het
- compleet is (meestal)
- voor geheugen en rekentijd even ‘slecht’ is als breedte-eerst (zelfs slechter want de sortering komt erbij!)

De afleiding van deze eigenschappen voor dit algoritme moet je niet kennen.

Vraag is nu natuurlijk hoe we dit algoritme verder kunnen verbeteren.

How to improve?

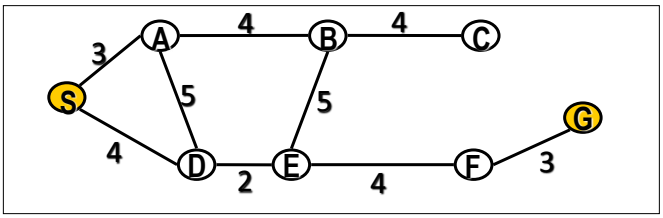
- Extend the algorithm with heuristic estimates
- Replace the 'accumulated cost' in the 'extended uniform cost' algorithm by a function:
  - f(path) = cost(path) + h(endpoint_path)
- Where:
  - cost(path) = the accumulated cost of the partial path
  - h(t) = a heuristic estimate of the cost remaining from T to a goal node

In het eerste en tweede algoritme in dit topic, uniform cost en optimal uniform cost, selecteerden we telkens het pad dat tot nu toe de laagste kost had om verder te zoeken naar het doel.
Dit was een naïeve keuze die er bij uniform cost voor zorgt dat het optimale pad niet gevonden wordt. Bij optimal uniform kost blijven we daarom verder zoeken na het vinden van een pad, wat dan weer ten koste gaat van extra rekentijd.

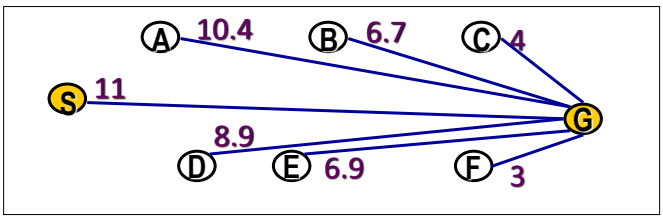
Het grote probleem bij het enkel kijken naar de geaccumuleerde kost is dat er geen rekening gehouden wordt met wat er nog komt op het pad. We kijken alleen achteruit, naar de start, maar niet vooruit, naar de goal. In topic 6 hebben we echter een techniek gezien om de 'kost' of 'afstand' vanuit een knoop naar het doel in te schatten, nl. de heuristische functie h! We willen vanaf nu deze inschatting gebruiken in het algoritme om het meest beloftevolle pad te selecteren om verder te analyseren in het algoritme.

We introduceren de f-waarde van een knoop, volgens de formule op de slide. De f-waarde van een knoop is de kost van de startnode tot aan die knoop waarbij je de inschatting van de kost van die knoop tot het doel optelt. Hoe lager dus de f-waarde, hoe meer kans dat een pad via die knoop tot het optimale pad tot het doel leidt. Let wel: dit is een kans, geen zekerheid!!!

Estimate-extended uniform cost algorithm



- h(T) = the straight-line distance from T to G



Bovenaan zie je de kost, onderaan de heuristiek. De f-waarde van een node is dus de som van de geaccumuleerde kost in die node en de heuristiek van die node.

Estimate-extended uniform cost algorithm

1. QUEUE ← path only containing the root;

2. WHILE (QUEUE is not empty AND first path does not reach goal)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the QUEUE and sort the entire QUEUE (by f = cost + h)

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

Het estimate-extended uniform cost algoritme is het optimale uniform cost algoritme, maar i.p.v. de QUEUE te sorteren op geaccumuleerde kost, sorteren we op f-waarde (van klein naar groot).

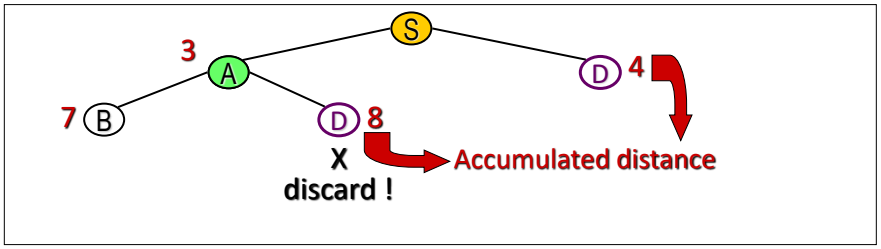
Estimate-extended uniform cost algorithm

- Complete and optimal:
  - Yes, for a good heuristic function h
  - Why? Believe me ;-)
- Speed and memory:
  - In the worst case: no improvement over 'branch and bounded extended uniform cost (optimal uniform cost)'
    - Just take h = 0 everywhere
  - For good heuristic function: search may expand much less nodes
- Can we still do better?

Ook hier, enkel resultaat kennen, analyse niet.

How to do better?

- Discard redundant paths
- Example



- Principle
  - The minimum distance from S to G via I =
    (min. dist. from S to I) + (min. dist. from I to G)

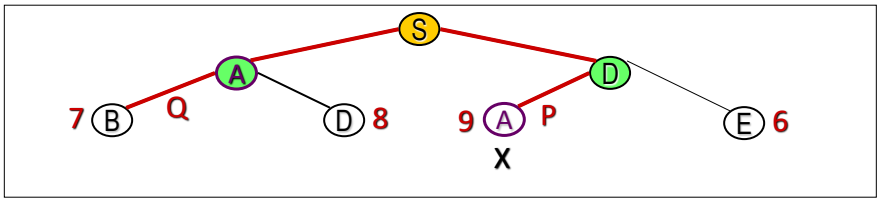
Zoals reeds aangegeven bij de uitleg over branch-and-bound moeten we op zoek gaan naar redundante paden.

Als we kijken naar de figuur, dan zien we vanuit S twee paden naar knoop D:
- een rechtstreeks pad met kost 4
- een pad langs knoop A met kost 8

Stel dat er via knoop D een pad naar het doel G bestaat (het doel staat niet op deze figuur, maar dat is niet nodig), dan zal het optimale pad naar G dat via D verloopt nooit langs knoop A passeren, want er bestaat een pad van S naar D met een lagere kost, nl. het pad dat rechtstreeks van S naar D gaat. Het pad naar D via A is dus eigenlijk redundant, en kan genegeerd worden.

Dit principe gieten we in een formule op de volgende slide, en willen we inbouwen in het algoritme dat we tot nu toe hebben.

How to do better?



- More precisely

IF the QUEUE contains:

- a path P terminating in I, with cost cost_P
- a path Q containing I, with cost cost_Q
- cost_P ≥ cost_Q

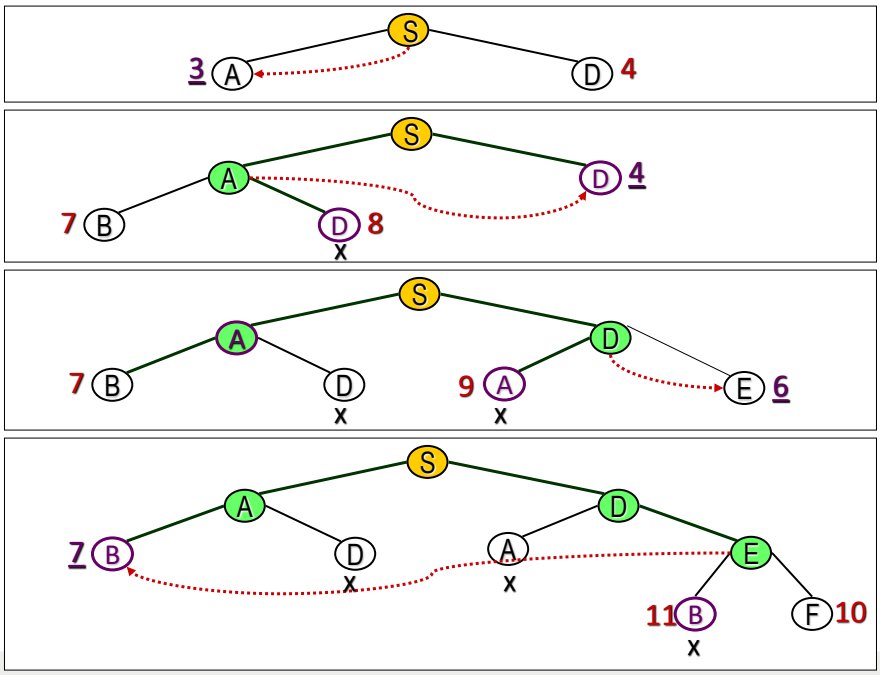
THEN

delete P

Pad Q (SAB) heeft kost 7
Pad P (SDA) heeft kost 9

Er is dus een pad dat verder gaat dan A en kost 7 heeft. Het pad dat naar A toegaat met grotere kost (9), kan dus nooit tot het optimale pad leiden, dus mag pad P weg.

Remove redundant paths: example



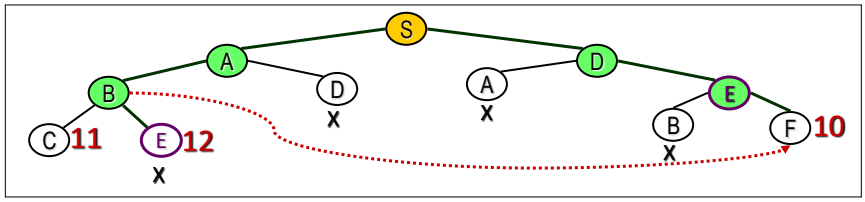
Probeer als oefening voor alle X'en in deze en de volgende slide de juiste formule (regel) op te stellen.

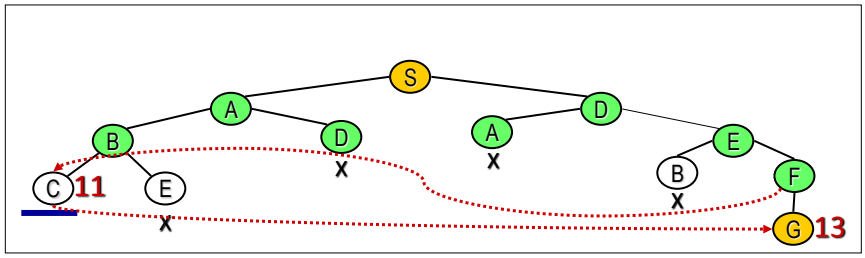
Voorbeeld:
SAD met kost 8 is redundant, want er is een pad SD met kost 4.

Oplossing:
- SAD met kost 8 is redundant, reden: pad SD met kost 4
- SDA met kost 9 is redundant, reden: pad SA met kost 3
- SDEB met kost 11 is redundant, reden: pas SAB met kost 7

Remove redundant paths: example

- Note how this optimization reduces the number of expansions VERY much, compared to ‘branch and bound extended uniform cost’.





- 5 expansions less!

Oplossing:
- SABE met kost 12 is redundant, reden: pad SDE met kost 6

A* algorithm

- Is a UNIFORM COST algorithm with:
  - Branch and bound extention
  - Estimate extention
  - Redundant path deletion extention

Het A* algoritme is zoals op de slide staat het uniform cost algoritme, uitgebreid met **alle** technieken die we in dit topic gezien hebben.

A* algorithm

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND first path does not reach goal)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths to the QUEUE and sort the entire QUEUE (by f = cost + h)
   - If QUEUE contains:
     - path P terminating in I, with cost_P
     - path Q containing I, with cost_Q

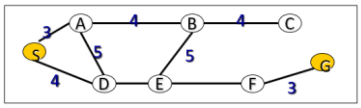
     and if cost_P ≥ cost_Q THEN delete path P

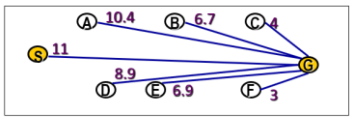
3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

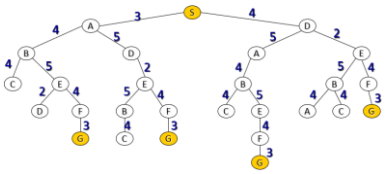
We vinden idd alle besproken extensies terug in de pseudocode A*:

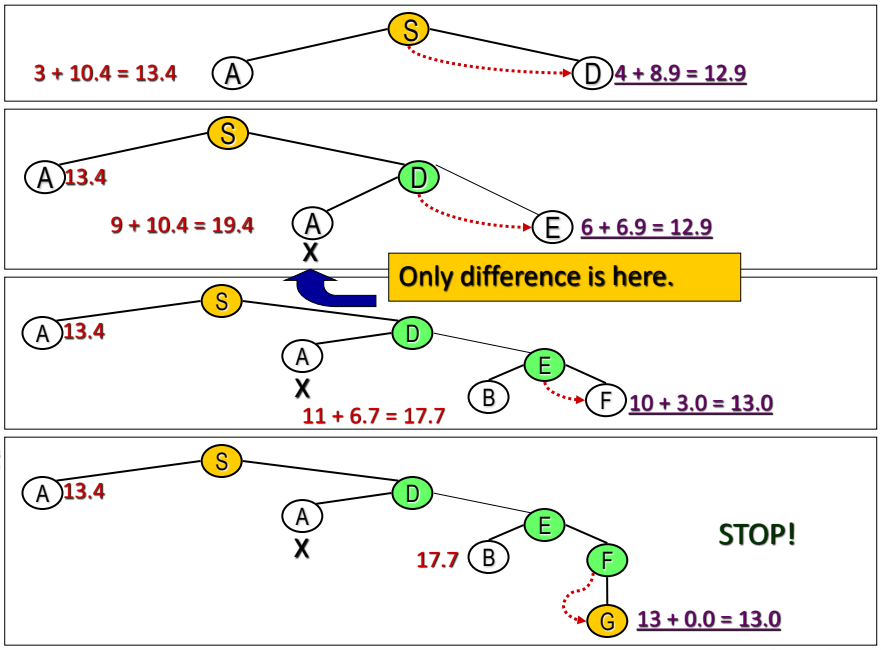
- Het verder zoeken tot het eerste pad in de queue de goal bereikt (weak integration of branch-and-bound)
- Het sorteren van de queue obv f-waarde (estimate-extended)
- Het verwijderen van redundante paden (redundant path deletion, in geel)

A* algorithm: example









Een voorbeeld uitvoering van het A* algoritme, waar de straight-line distance als heuristiek (functie h) gebruikt wordt.

Questions

?

Coordinates:

Dr. Stefaan Haspeslagh

A.I. Research & Education

Vives University of applied sciences – KU Leuven Association

E-mail: stefaan.haspeslagh@vives.be